Week-13

Introduction to Database Systems

Shah Nawaz

Functional Dependencies
Inference Rules,
Closure of Attributes,
Identifying Keys from Functional Dependencies

Functional Dependencies

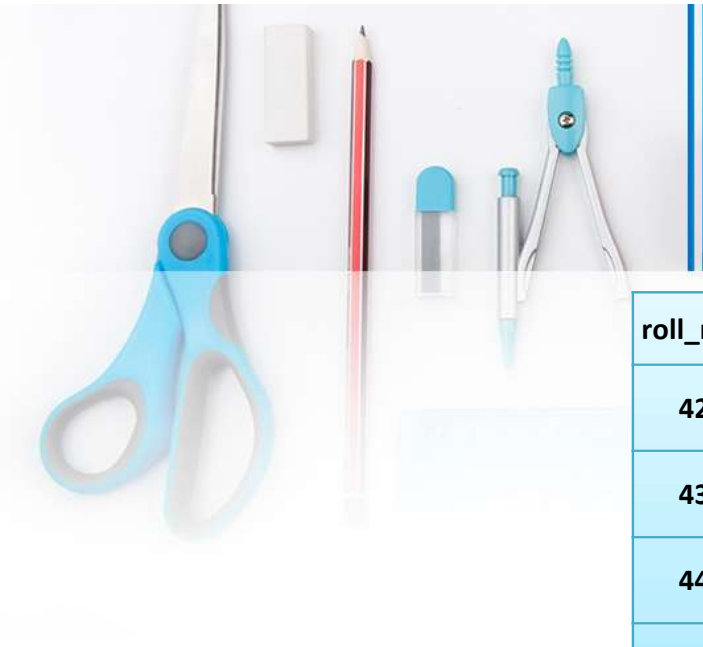
In a relational database management, functional dependency is a concept that specifies the relationship between two sets of attributes where one attribute determines the value of another attribute.

It is denoted as

$$X \rightarrow Y$$

where the attribute set on the left side of the arrow, **X** is called **Determinant**, and **Y** is called the **Dependent**.

Functional dependencies are used to mathematically express relations among database entities and are very important to understand advanced concepts in Relational Database System.



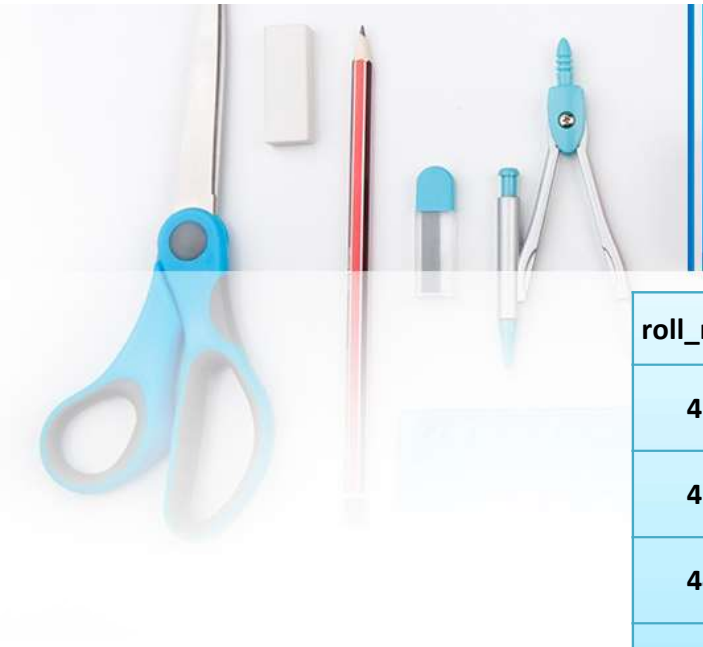
Functional Dependencies

roll_no	name	dept_name	dept_building
421	abc	CO	A4
432	pqr	IT	A3
441	xyz	CO	A4
452	xyz	IT	A3
463	mno	EC	B2

From the above table we can conclude some Valid functional dependencies:

- roll_no → { name, dept_name, dept_building }, Here, roll_no can determine values of fields name, dept_name and dept_building, hence a valid Functional dependency

- roll_no → dept_name , Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.

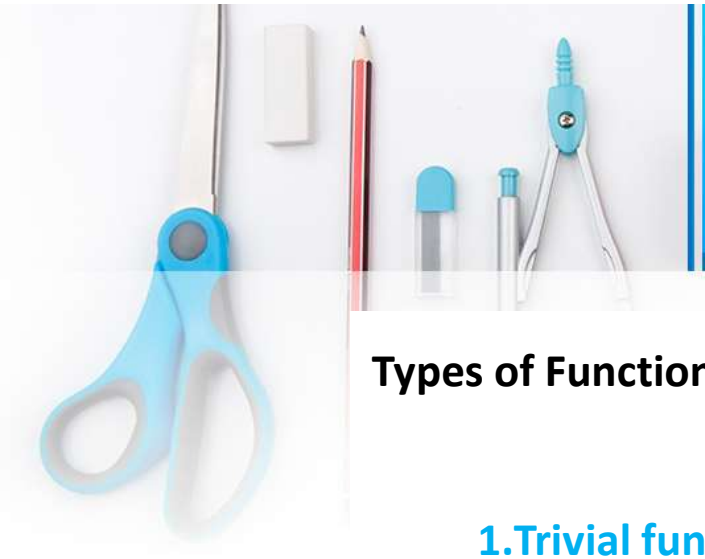


Functional Dependencies

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2

Here are some invalid functional dependencies:

- **name → dept_name** Students with the same name can have different dept_name, hence this is not a valid functional dependency.
- **dept_building → dept_name** There can be multiple departments in the same building. Example, in the above table departments ME and EC are in the same building B2, hence dept_building → dept_name is an invalid functional dependency.
- **More invalid functional dependencies:** name → roll_no, {name, dept_name} → roll_no, dept_building → roll_no, etc.



Functional Dependencies

Types of Functional Dependencies

1. Trivial functional dependency
2. Non-Trivial functional dependency
3. Multivalued functional dependency
4. Transitive functional dependency
5. Fully functional dependency
6. Partial functional dependency

Functional Dependencies

1.Trivial functional dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e.

If X → Y and Y is the subset of X, then it is called trivial functional dependency.

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **{roll_no, name} → name** is a trivial functional dependency, since the dependent **name** is a subset of determinant set **{roll_no, name}.** Similarly, **roll_no → roll_no** is also an example of trivial functional dependency.

Functional Dependencies

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

2. Non-Trivial functional dependency

In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant. i.e.
If **X → Y** and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

Here, **roll_no → name** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no.**

Similarly, **{roll_no, name} → age** is also a non-trivial functional dependency, since **age** is **not a subset of {roll_no, name}**

Functional Dependencies

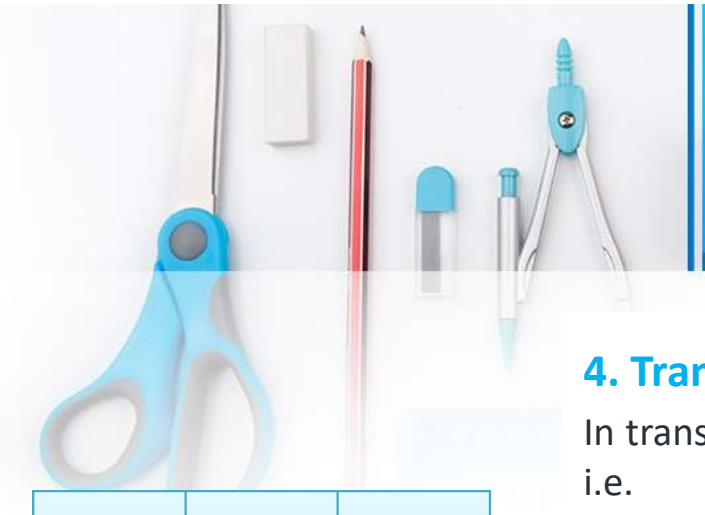
3. Multivalued functional dependency

In **Multivalued functional dependency**, entities of the dependent set are **not dependent on each other.** i.e.

If **a → {b, c}** and there exists **no functional dependency** between **b and c**, then it is called a **multivalued functional dependency.**

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **roll_no → {name, age}** is a multivalued functional dependency, since the dependents **name** & **age** are **not dependent** on each other(i.e. **name → age** or **age → name doesn't exist !**)



Functional Dependencies

4. Transitive functional dependency

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e.

If **a → b** & **b → c**, then according to axiom of transitivity, **a → c**. This is a **transitive functional dependency.**

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **roll_no → dept** and **dept → building_no.** Hence, according to the axiom of transitivity, **roll_no → building_no** is a valid functional dependency.

This is an indirect functional dependency, hence called Transitive functional dependency.

Functional Dependencies

5. Fully functional dependency

In full functional dependency an attribute or a set of attributes uniquely determines another attribute or set of attributes.

If a relation **R has attributes X, Y, Z** with the dependencies **X->Y** and **X->Z** which states that those dependencies are fully functional.

Functional Dependencies

6. Partial functional dependency

In partial functional dependency a non key attribute depends on a part of the composite key, rather than the whole key.

If a relation R has attributes X, Y, Z where X and Y are the composite key and Z is non key attribute. Then **X → Z** is a partial functional dependency in RBDMS.

```
CREATE TABLE StudentCourses (
   StudentID INT,
   CourseID INT,
   CourseName VARCHAR(50),
   Instructor VARCHAR(50),
   PRIMARY KEY (StudentID, CourseID)
);
```

In this example, **(StudentID, CourseID)** is the composite primary key. However, there is a partial functional dependency if the Instructor attribute depends **(StudentID, CourseID)** → **Instructor** only on a part of the key, let's say only on CourseID rather than the entire composite key.

Armstrong’s axioms Inference Rules

Armstrong's axioms are a set of references used to infer all the functional dependencies on a relational database. They were developed by William W. Armstrong in his 1974 paper. The axioms are sound in generating only functional dependencies in the closure of a set of functional dependencies when applied to that set.

Inference Rules

Inference rules in the context of SQL typically refer to rules that allow for deriving additional information or conclusions based on the existing data and the structure of the database. In SQL, such rules are often associated with logical implications or transformations that can be applied to database queries. Here are a few common inference rules used in SQL:

1. **Transitive Rule**
2. **Augmentation Rule**
3. **Addition Rule**
4. **Decomposition Rule**
5. **Union Rule**

Inference Rules

Transitive Rule:

If A implies B, and B implies C, then A implies C.

Example: If EmployeeID implies EmployeeName, and EmployeeName implies Department, then EmployeeID implies Department.

Augmentation Rule:

If A implies B, then A, C implies B, where C is any additional condition.

Example: If Department implies Manager, then Department and Location implies Manager.

Inference Rules

Addition Rule:

If A implies B, then A implies B or C.

Example: If Salary implies Bonus, then Salary implies Bonus or Commission.

Decomposition Rule:

If A implies B and C, then A implies B and A implies C.

Example: If EmployeeID implies EmployeeName and Department, then EmployeeID implies EmployeeName and EmployeeID implies Department.

Union Rule:

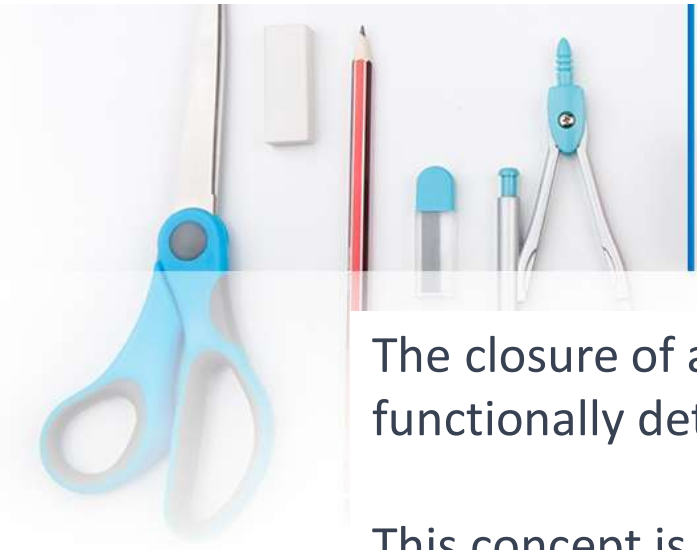
If A implies B and A implies C, then A implies B or C.

Example: If EmployeeType implies Salary and EmployeeType implies HourlyRate, then EmployeeType implies Salary or HourlyRate.

Inference Rules

Axiom Name	Axiom	Example
Reflexivity	if a is set of attributes, b ⊆ a, then a →b	SSN,Name → SSN
Augmentation	if a→ b holds and c is a set of attributes, then ca→cb	SSN → Name then SSN,Phone → Name, Phone
Transitivity	if a →b holds and b→c holds, then a→ c holds	SSN →Zip and Zip → City then SSN →City
Union or Additivity *	if a → b and a → c holds then a→ bc holds	SSN→Name and SSN→Zip then SSN→Name,Zip
Decomposition or Projectivity*	if a → bc holds then a → b and a → c holds	SSN→Name,Zip then SSN→Name and SSN→Zip
Pseudotransitivity*	if a → b and cb → d hold then ac → d holds	Address → Project and Project,Date →Amount thenAddress,Date → Amount
(NOTE)	ab→ c does NOT imply a → b and b → c	

Attribute Closure

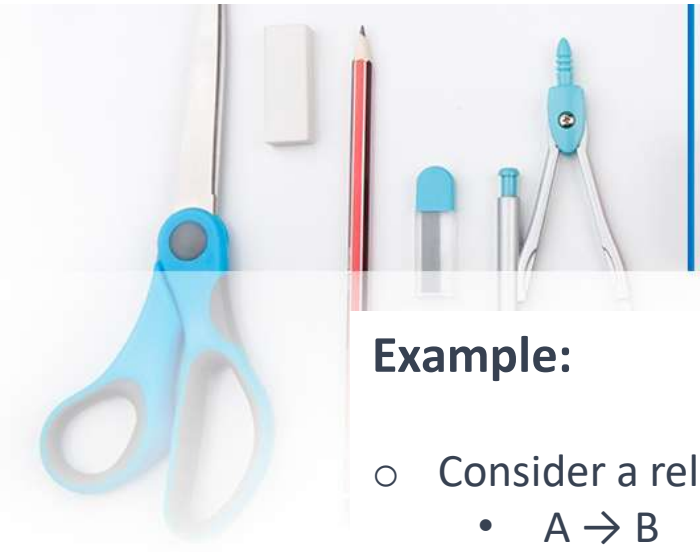


Attribute Closure

The closure of attributes refers to the set of all attributes that can be functionally determined, directly or indirectly, by a given set of attributes.

This concept is fundamental in the study of functional dependencies and normalization in database design.

Closure of any attribute **X** is denoted as $\mathbf{X^+}$ or **({X})**



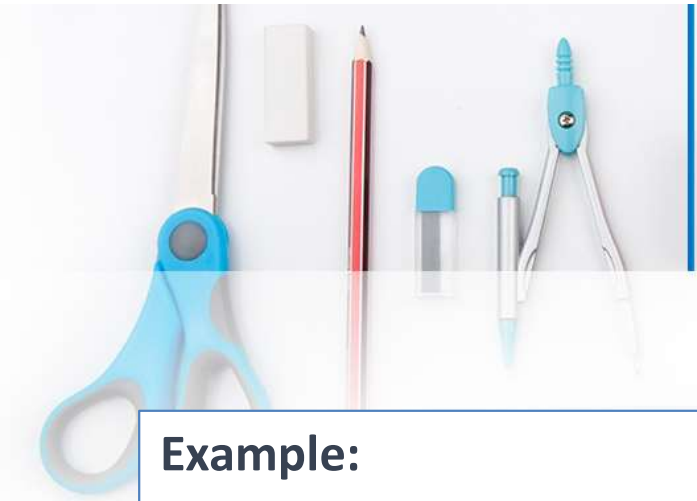
Attribute Closure

Example:

- Consider a relation R(A, B, C, D) with functional dependencies:
  - A → B
  - B → C
  - CD → A

- To find the closure of attributes for {A}, you would apply the dependencies to get Closure({A}) = {A, B, C}.

- To find the closure of attributes for {B, CD}, you would apply the dependencies to get Closure({B, CD}) = {B, C, D, A}.



Attribute Closure

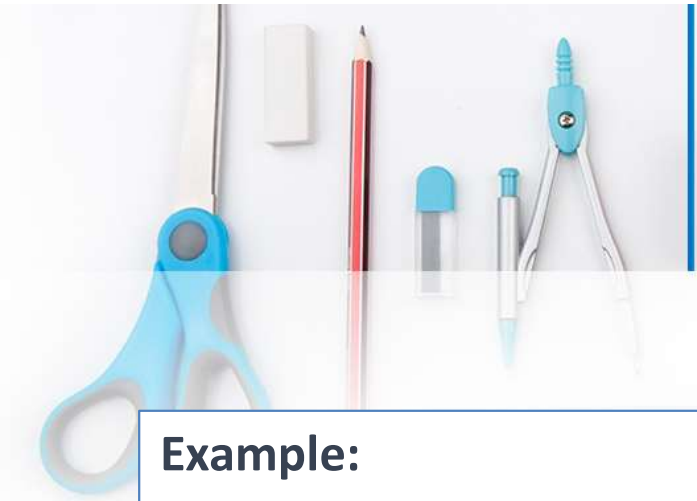
Example:

- Consider a relation R(A, B, C, D, E, F, G) with functional dependencies:
  - A → B
  - BC → DE
  - AEG → G
- Find $(AC)^+$

Solution:

- $(AC)^+ = AC$
- $= ABC$
- $= ABCDE$

Answer = ABCDE



Attribute Closure

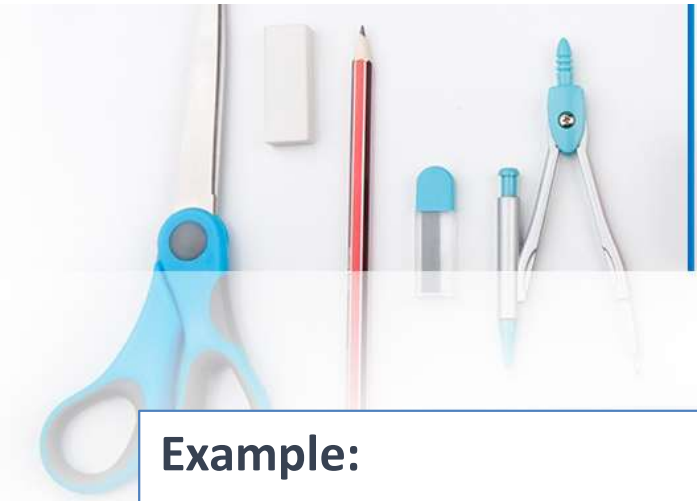
Example:

- Consider a relation R(A, B, C, D, E) with functional dependencies:
  - A → BC
  - CD → E
  - B → D
  - E → A
- Find $(B)^+$

Solution:

- $(B)^+ = B$
- $= BD$

Answer = BD



Attribute Closure

Example:

- Consider a relation R(A, B, C, D, E, F, G,H) with functional dependencies:
  - A → BC
  - CD → E
  - E → C
  - D → AEH
  - ABH → BD
  - DH → BC
- Find BCD →H

Solution:

- $(BCD)^+ = BCD$
- $=BCDE$
- $=ABCDEH$

Answer = ABCDEH

Identifying Keys from Functional Dependencies

Sample Table

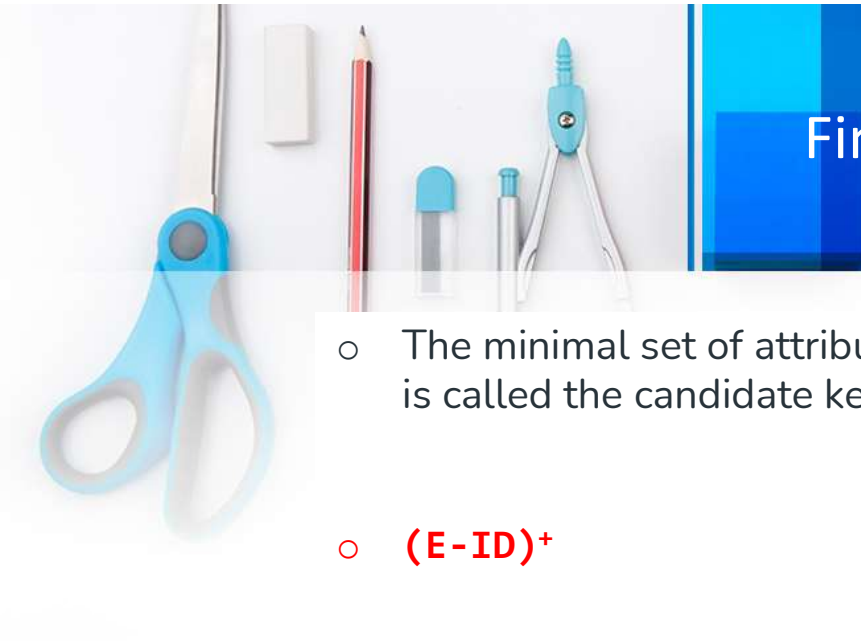
E-ID	E-NAME	E-CITY	E-STATE
E001	John	Lhr	Punjab
E002	Mary	Lhr	Punjab
E003	John	Karachi	Sindh

Finding Super Keys of a Relation using FD set

- The **set of attributes** whose attribute closure is a set of all attributes of the relation is called the super key of the relation.
- For Example, the EMPLOYEE relation shown in Table 1 has the following FD set.
  EMPLOYEE {E-ID->E-NAME, E-ID->E-CITY, E-ID->E-STATE, E-CITY->E-STATE}
- Let us calculate the attribute closure of different sets of attributes:

- $(E\text{-}ID)^+ = \{E\text{-}ID, E\text{-}NAME, E\text{-}CITY, E\text{-}STATE\}$
  $(E\text{-}ID, E\text{-}NAME)^+ = \{E\text{-}ID, E\text{-}NAME, E\text{-}CITY, E\text{-}STATE\}$
  $(E\text{-}ID, E\text{-}CITY)^+ = \{E\text{-}ID, E\text{-}NAME, E\text{-}CITY, E\text{-}STATE\}$
  $(E\text{-}ID, E\text{-}STATE)^+ = \{E\text{-}ID, E\text{-}NAME, E\text{-}CITY, E\text{-}STATE\}$
  $(E\text{-}ID, E\text{-}CITY, E\text{-}STATE)^+ = \{E\text{-}ID, E\text{-}NAME, E\text{-}CITY, E\text{-}STATE\}$
  $(E\text{-}NAME)^+ = \{E\text{-}NAME\}$
  $(E\text{-}CITY)^+ = \{E\text{-}CITY, E\text{-}STATE\}$

- As $(E\text{-}ID)^+$, $(E\text{-}ID, E\text{-}NAME)^+$, $(E\text{-}ID, E\text{-}CITY)^+$, $(E\text{-}ID, E\text{-}STATE)^+$, $(E\text{-}ID, E\text{-}CITY, E\text{-}STATE)^+$ give set of all attributes of relation EMPLOYEE. So all of these are super keys of relation.



Finding Candidate Keys of a Relation using FD set

- The minimal set of attributes whose attribute closure is a set of all attributes of relation is called the candidate key of the relation.

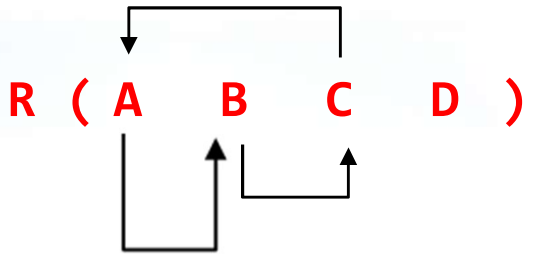
- $(E\text{-}ID)^+ = \{E\text{-}ID, E\text{-}NAME, E\text{-}CITY, E\text{-}STATE\}$

- As shown above, (E-ID)+ is a set of all attributes of relation and it is minimal.
- So E-ID will be the candidate key. On the other hand (E-ID, E-NAME)+ also is a set of all attributes but it is not minimal because its subset (E-ID)+ is equal to the set of all attributes. So (E-ID, E-NAME) is not a candidate key.

Finding Candidate Keys of a Relation using FD set

- **R (ABCD)**

- A → B
- B → C
- C → A



Step 1: Draw edge diagram to graphically represent FDs (This step is optional)
Step 2: Select those attribute (s) which are not on the right side of FDs (This is D)
Step3: Find Closure of $(D)^+$, if this closure contains all the attributes or R then D is Candidate Key.
Step 4: Otherwise make combinations of D with all other attributes e.g.
AD, BD, CD and find closures of all these combination $(AD)^+$, $(BD)^+$, $(CD)^+$ according to give FDs.
Step: Any closure which contains all the attributes of R, is a candidate key

Note: Any Relation may contain one or more than one candidate keys

Thank You all!